MyCodeSchool

Code review — recapitulare-initiere-cpp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 4 — 2026-10-15 — sha 6c66dca

> **Runda 3 n-a ajuns la tine.** În `65ec85f` ai comis peste `CODE_REVIEW.pdf` PDF-ul rundei 2, cu notițele tale. Runda 3 a rămas la mine. O găsești mai jos, condensată: aproape tot ce era în ea ai reparat singur între timp. Cele două constatări rămase deschise (B15, B16) sunt explicate din nou aici, în runda 4.

Patru commit-uri de la ultima rundă: `65ec85f`, `acf2773`, `8762a7b`, `6c66dca`. Ai lucrat pe fișa de 75 de probleme, secțiunile 1–5, câte un header pe fiecare algoritm, exact cum cerea fișa.

Am compilat **fiecare header separat** și am rulat **fiecare funcție pe exemplele din fișă**, plus capetele (vector cu valori egale, valoare mai mică decât toate, valoare mai mare decât toate). Pe ce am rulat am folosit AddressSanitizer: un instrument care oprește programul în clipa în care citește în afara vectorului. Nimic de mai jos nu e dedus din citit.

Poarta de compilare: 🔴 pică. `main.cpp` compilează, fiindcă include doar `sortare_interval.h`. Dar `algoritmi.h` nu mai compilează din `6c66dca`, și odată cu el pică **toate cele 4 headere care îl includ**: `cautare_binara.h`, `minim_maxim.h`, `sortare_bule.h`, `sortare_selectie.h`. Vezi B17.

Bilanțul fișei, secțiunile 1–5 (25 de exerciții):

	Exerciții
✅ corecte, prin rulare	1.1 · 1.2 · 1.3 · 2.2 · 3.1 · 3.2 · 3.5 · 4.1 · 5.1 · 5.4 — **10**
🔴 rezultat greșit	1.4 · 2.1 · 2.3 · 2.4 · 3.4 · 4.2 · 4.4 · 5.2 · 5.3 · 5.5 — **10**
⏳ neterminate	1.5 · 2.5 · 3.3 · 4.3 · 4.5 — **5**

Ce s-a închis din rundele trecute

Ai reparat regula, nu doar linia. E prima rundă în care se întâmplă.

#	Stare	Verificat prin
B2 (r1) — cifra scoasă înainte de `while`	✅ **ÎNCHIS, în toate cele 3 locuri**	`verifcifre(8123,8)` → 1 · `cifraMaxima(8123)` → 8 · `areToateCifrelePrime(4357)` → 0
B5 (r1) — `cmmdc(0,0)`	✅ **ÎNCHIS**	`cmmdc(0,0)` → 0 · `(0,5)` → 5 · `(12,18)` → 6 · `(7,13)` → 1
B7 (r1) — `celMaiMicNrCuSumaCifrelor(20)`	✅ **ÎNCHIS** (în funcția nouă)	`(20)` → 299 · `(10)` → 19 · `(27)` → 999 · `(0)` → 0
B9 (r2) — `contorPrimeVector` număra indicii	✅ **ÎNCHIS**	`{23,5,4,3,7}` → 4 · `{10,12,14}` → 0
B11 (r2) — palindromul testa doar capetele	✅ **ÎNCHIS**	`punctul_c` pe `{12,323,45,65,1231}` → doar `323`
B14 (r2) — `afisareaElemVect` era `int`	✅ **ÎNCHIS**	e `void` acum
B15 (r3) — `produsCifre`	🔴 **NEATINS**	`produsCifre(234)` → 4, trebuia 24
B16 (r3) — „prime între ele”	🔴 **NEATINS**	`:223` tot `isPrim(i)`

B2 era o greșeală în trei funcții. Data trecută ai fi reparat-o doar pe cea despre care vorbeam. Acum ai scris același tipar corect în toate trei:

```
while(n > 0)
{
    int uc = n % 10;
    ...
    n = n / 10;
}
```

Asta e diferența dintre „am reparat ce mi s-a zis" și „am înțeles ce era greșit". De aici încolo, te rog să faci la fel cu fiecare constatare.

🔴 Critice

B17 — `algoritmi.h` nu mai compilează, iar cu el pică 4 headere

`algoritmi.h:169` · `cautare_binara.h:190–207`

CUM E ACUM:

```
algoritmi.h:169:28: error: use of undeclared identifier 'aux'
```

Ai modificat `sortareIntervalDescrescator` din `algoritmi.h`:

```
v[i + 1] = aux;        // linia mutată de tine
int aux = v[i];
v[i] = v[i + 1];
```

`cautare_binara.h`, `minim_maxim.h`, `sortare_bule.h` și `sortare_selectie.h` au toate `#include "algoritmi.h"` sus. Deci niciunul nu mai compilează, deși în ele n-ai schimbat nimic. Ca să-ți pot rula funcțiile, am pus în locul lui `algoritmi.h` versiunea de la `8762a7b`, care compila.

Plus, `cautare_binara.h` are deja 3 erori proprii, de la `8762a7b`:

```
cautare_binara.h:203: error: excess elements in scalar initializer
cautare_binara.h:206: error: use of undeclared identifier
    'parcurgereInVector'
```

`int v = {2, 5, 9, 14, 20};` declară un singur `int`, nu un vector. Iar `parcurgereInVector` nu există nicăieri.

DE CE: `#include` nu înseamnă „folosește fișierul ăla". Înseamnă „**copiază textul lui aici**, înainte să compilezi". Când scrii `#include "algoritmi.h"`, compilatorul vede cele 600 de linii lipite sus în fișierul tău, ca și cum le-ai fi scris tu. Dacă una dintre ele e stricată, e stricat și fișierul tău. Așa se face că o singură linie mutată în `algoritmi.h` a stricat 4 headere la care nici nu te-ai atins.

Și ăsta e motivul pentru care `algoritmi.h` e **material de referință**: toate exercițiile tale se sprijină pe el. Îl poți citi, poți trasa pe el, poți chema din el orice. Dar nu-l modifici: dacă vrei să încerci o variantă, o scrii în headerul tău.

Din cele 4 commit-uri, **două nu compilau nici ele**: `acf2773` (5 erori) și `8762a7b` (3 erori). Ai lucrat pe branch în runda trecută și ți-a ieșit foarte bine. Un exercițiu neterminat stă pe branch până compilează, sau îl comentezi.

FIX: pune înapoi `v[i + 1] = aux;` după `v[i] = v[i + 1];` în `algoritmi.h` (vezi B18 pentru de ce). În `cautare_binara.h`, comentează `cautarePoz` și `sol25` până le termini. Apoi, înainte de fiecare commit, include **pe rând** fiecare header în `main.cpp` și apasă F9.

B18 — interschimbarea în ordinea greșită distruge vectorul (5.2, 5.3)

`sortare-interval.h:61` · `:70-71`

CUM E ACUM: rulat pe exemplul din fișă, `9 7 5 3 1 2`:

```
5.2  sortareAdouaJumatateDesc
     rezultat:  9 7 5 6 6 6
     trebuia:   9 7 5 3 2 1

5.3  sortareJumatate
     rezultat:  5 7 9 16777216 16777216 16777216
     trebuia:   5 7 9 3 2 1
```

`3`, `1` și `2` au **dispărut** din vector. În locul lor apar numere pe care nu le-ai scris nicăieri. Și diferă de la o rulare la alta.

DE CE, în doi pași.

1. Ordinea celor trei atribuiri contează. O interschimbare are nevoie de un loc unde să păstreze o valoare, până nu o pierde:

```
aux = v[i]          păstrez v[i]
v[i] = v[i + 1]     acum pot scrie peste v[i]
v[i + 1] = aux      pun înapoi ce am păstrat
```

Dacă scrii `v[i + 1] = aux` **primul**, scrii peste `v[i + 1]` înainte să-l fi păstrat undeva. Valoarea lui s-a pierdut, și nimic nu o mai aduce înapoi.

2. Ce a făcut eroarea să dispară. În `algoritmi.h`, compilatorul te-a oprit: `use of undeclared identifier 'aux'`, fiindcă `aux` era folosit cu o linie înainte de a fi declarat. Aici, în copia ta, ai pus `int aux;` sus, la linia 61. Eroarea a dispărut, dar **bug-ul a rămas, doar că acum nu-l mai semnalează nimeni**.

Acum ai **două** variabile cu numele `aux`: una la `:61`, care nu primește niciodată o valoare, și una la `:71`, declarată în `if`, care o ascunde pe prima. Linia `:70` o citește pe cea de la `:61`, adică ce se nimerește să fie în memorie în acel moment. De-aia vezi `16777216` și `6`: sunt resturi din memorie, nu valori din vectorul tău.

Compilatorul ți-a spus asta din nou, ca warning:

```
sortare-interval.h:70: warning: variable 'aux' is uninitialized
    when used here
sortare-interval.h:71: warning: unused variable 'aux'
```

Regula de reținut: **o eroare de compilare îți spune ce e greșit.** Dacă o faci să tacă fără să înțelegi ce spunea, bug-ul rămâne și doar nu-l mai vezi.

FIX: șterge `int aux;` de la `:61` și pune cele trei linii în ordinea din tabelul de mai sus. Apoi rulează din nou 5.2 și 5.3.

B19 — „funcție `int` fără `return`", a patra rundă la rând

`cautare_binara.h:90` · `:108` · `:190` · `minim_maxim.h:62` · `sortare_bule.h:43` · `sortare_selectie.h:24` · `:44`

CUM E ACUM: șapte funcții declarate `int` ajung la `}` fără `return`. Trei dintre ele le-am prins în rulare:

```
2.3  pozitiaDeInserat(2 5 9 14, pun 20)
     runtime error: execution reached the end of a value-returning
     function without returning a value
1.4  aDouaValoare(7 2 9 4 9)     -> aceeași eroare
4.2  ordonareDupaUc(23 41 15 32) -> aceeași eroare
```

Fără AddressSanitizer, `sol4()` afișează `0`. Nu e o valoare calculată de funcția ta. E ce se nimerește să fie în locul unde ar fi trebuit să stea rezultatul.

DE CE: când scrii `int f(...)`, îi promiți celui care cheamă funcția că primește înapoi un `int`. Valoarea pleacă abia la `return`. Dacă funcția ajunge la `}` fără `return`, cel care a chemat-o citește oricum din locul unde ar fi trebuit să fie rezultatul. Acolo nu a scris nimeni nimic, deci primește un rest de la un calcul de mai devreme. Standardul C++ spune că de aici încolo programul nu mai are niciun comportament garantat.

Ai deja regula: în runda 3 ai făcut `void` cele 4 funcții care doar afișează. Aici sunt **două cazuri diferite**: - **funcția doar afișează** (`ordonareDupaUc` , `afisareDistinceDescrescatoare` , `ordonarePare`) → trebuie `void` ; - **funcția trebuie să dea un răspuns** (`pozitiaDeInserat` , `aDouaValoare`) → îi lipsește `return` pe drumul în care bucla se termină fără să găsească nimic. La `pozitiaDeInserat` , dacă valoarea e mai mare decât toate, răspunsul e `dim` . Exemplul 3 din fișă (`pun 20 -> 4`) testează exact cazul ăsta.

Code::Blocks ți-a spus de fiecare dată, la F9: `warning: no return statement in function returning non-void` .

FIX: caută în toate headerele cuvântul `int` la începutul unei funcții și verifică dacă **fiecare drum** prin ea se termină cu `return` . Nu doar în cele 7 de aici. În `while-ex.h` au rămas încă două, la `:124` și `:169` .

B20 — 2.4: `<` în loc de `<=` , iar forma corectă e scrisă de tine la 30 de linii

`cautare_binara.h:142` · `:145`

CUM E ACUM: rulat pe `2 5 9 14`:

```
x = 6   ->  9     corect (exemplul din fișă)
x = 15  -> -1     corect (exemplul din fișă)
x = 9   -> -1     trebuia 9
x = 14  -> -1     trebuia 14
x = 1   ->  5     trebuia 2
x = 2   ->  5     trebuia 2
```

Exemplele din fișă trec amândouă. Toate celelalte cazuri pe care le-am încercat sunt greșite.

DE CE: sunt două greșeli, fiecare cu efectul ei.

- **`while(minim < maxim)`** se oprește când `minim == maxim`, adică exact când a mai rămas **un singur element de verificat**. Elementul acela nu mai e verificat niciodată. De-aia `x = 1` dă `5`: căutarea a ajuns la `v[0] = 2` și s-a oprit fără să se uite la el.
- **`v[mij] > key`** sare peste valoarea egală. Cerința spune „mai mare **sau egală**". De-aia `x = 9` dă `-1`, deși `9` e în vector.

Iar acum partea importantă. **La linia 175, în `cautarePrimaPoz`, ai scris fix forma corectă:**

```
while(minim <= maxim)
{
    int mij = (minim + maxim) / 2;
    if(v[mij] >= x)
```

E a patra rundă în care forma corectă și cea greșită stau în același fișier. `cautarePrimaPoz` întoarce poziția celei mai mici valori `>= x`, adică exact ce cere 2.4. Doar că 2.4 cere valoarea, nu poziția.

FIX: nu repara bucla din `ceaMaiMicaValoare`, ci folosește funcția corectă pe care o ai deja:

```
int poz = cautarePrimaPoz(v, dim, key);
```

Dacă `poz < dim`, răspunsul e `v[poz]`; altfel e `-1`. Și mută `cautarePrimaPoz` deasupra lui `ceaMaiMicaValoare`, ca să o poți chema de acolo.

B21 — 2.1: căutarea binară pe un vector neordonat

`cautare_binara.h:48` · `:57`

CUM E ACUM: pe `30 12 45 7 19`, cu `dim = 5`:

```
caut 19  ->  0     trebuia 1   (exemplul din fișă)
caut 7   ->  0     trebuia 1
caut 12  ->  0     trebuia 1
caut 30  ->  1     corect
caut 45  ->  1     corect
caut 20  ->  0     corect
```

În `sol21` ai căutat `20`, care **nu e** în vector. Răspunsul așteptat e `false`, deci testul trece. E singurul caz din fișă care trece oricum, fiindcă nu e nimic de găsit. Plus, `sol21` are `dim = 4` pentru 5 valori, deci `19` nici nu intră în căutare.

DE CE: căutarea binară se uită la mijloc și **aruncă o jumătate** întreagă fără s-o mai verifice. Are voie s-o arunce doar pentru că vectorul e ordonat: dacă `v[mij] < key`, tot ce e la stânga e și mai mic, deci acolo nu are ce căuta. Pe `30 12 45 7 19`, la mijloc e `45`, iar `19 < 45`, deci aruncă dreapta. Dar `19` era chiar în dreapta. Pe un vector neordonat, „la stânga e mai mic" nu mai e adevărat, și căutarea aruncă exact jumătatea în care era valoarea.

De-aia fișa scrie „vectorul primit **nu este neapărat ordonat**". E capcana exercițiului: întâi ordonezi, abia apoi cauți.

FIX: cheamă întâi `sortareCrescator(v, dim)` din `algoritmi.h`, apoi `cautareBinara`. Și în `sol21`, `dim = 5`.

B22 — 1.4: compari o valoare cu o poziție

`minim_maxim.h:64–69`

CUM E ACUM:

```
aDouaValoare(7 2 9 4 9)  ->  runtime error (fără return, vezi B19)
aDouaValoare(3 3 8 1)    ->  3     corect... din întâmplare
```

DE CE: `pozMaxim` întoarce **poziția** maximului, nu **valoarea** lui. Pe `3 3 8 1`, `pozMaxim` dă `2` (unde stă `8`), nu `8`. Linia `:67` compară deci fiecare element cu `2`. Primul element mai mic decât `2` e `1`, aflat la poziția `3`, iar funcția întoarce **poziția** lui, `3`.

Răspunsul corect e tot `3`, dar din alt motiv: e a doua valoare. Coincidența face să pară că funcția merge. Pe exemplul celălalt din fișă, niciun element nu e mai mic decât `2`, deci bucla se termină fără `return`.

Aceeași confuzie apare și în `amplitudinea`, dar acolo ai scris-o corect: `v[pozMaxim(v, dim)]`, cu `v[...]` în jur. Ai pus `v[...]` acolo, dar nu și aici.

FIX: `int maxx = v[pozMaxim(v, dim)];`. Apoi caută **cea mai mare valoare care e strict mai mică decât** `maxx`. Nu prima întâlnită, ci cea mai mare dintre ele. Cazul `4 4 4`, unde nu există a doua valoare, hotărăște tu ce întoarce și scrie-o într-o linie de comentariu deasupra funcției.

B23 — 3.4: funcția care întreabă „e ordonat?" îl ordonează

`sortare_bule.h:62–67`

CUM E ACUM:

```
dejaOrdonat(1 3 7)  ->  1    corect
dejaOrdonat(1 7 3)  ->  0    corect
vectorul după apel:     1 3 7      era 1 7 3
```

Răspunsul e corect, dar vectorul cu care ai chemat funcția **nu mai e cel de dinainte**.

DE CE: ai făcut o copie în `aux`, foarte bine. Dar apoi ai sortat **originalul**, `v`, nu copia. Iar `v` nu e un vector nou, făcut pentru funcție. Când trimiți un vector unei funcții în C++, ea primește **adresa** vectorului tău, nu o copie. Orice schimbi în `v` înăuntru se vede și în afară, după apel.

Cerința zicea explicit „**fără să-l sortezi**". Gândește-te la cine folosește funcția: întreabă „e ordonat?", iar după răspuns vectorul lui e altul. O funcție care doar răspunde la o întrebare nu are voie să schimbe nimic.

Și sortarea nici nu e necesară. Un vector e ordonat crescător dacă **nicio pereche de vecini** nu e invers.

FIX: o singură buclă: `for(i = 0; i < dim - 1; i++)`, iar dacă `v[i] > v[i + 1]`, întorci `false`. Dacă bucla se termină, întorci `true`. Fără copie, fără sortare.

B24 — 4.2: afișezi ultimele cifre, nu numerele

`sortare_selectie.h:26–35`

CUM E ACUM:

```
ordonareDupaUc(23 41 15 32)  ->  1 2 3 5        trebuia 41 32 23 15
```

DE CE: ai pus ultimele cifre într-un vector separat, `aux`, și ai sortat `aux`. Dar după ce ai sortat cifrele, **nu mai știi cărui număr îi aparține fiecare**. `1` venea de la `41`, dar nimic din `aux` nu mai ține minte asta. Ai sortat cheile, dar ai pierdut legătura cu numerele.

Cheia (ultima cifră) e doar criteriul **după care** compari. Ce muți în vector sunt tot numerele întregi.

FIX: sortarea prin selecție pe `v`, dar cu comparația `v[j] % 10 < v[min] % 10` în loc de `v[j] < v[min]`. Muți numerele, compari ultimele cifre. E singurul loc din fișă unde ai voie să-ți scrii propria sortare, fiindcă `sortareSelectie` din `algoritmi.h` compară valorile întregi.

B25 — 4.4: `v[i - 1]` la `i = 0` citește în afara vectorului

`sortare_selectie.h:49`

CUM E ACUM: `sol44()` afișează `2 5 7`, adică răspunsul corect. Cu AddressSanitizer:

```
ERROR: AddressSanitizer: stack-buffer-underflow
READ of size 4
```

DE CE: la primul pas al buclei, `i = 0`, deci `v[i - 1]` e `v[-1]`. Poziția `-1` nu e în vectorul tău: e memoria **dinaintea** lui, care aparține altcuiva. Rezultatul e corect pentru că valoarea găsită acolo s-a nimerit să fie diferită de `v[0]`. Cu altă valoare acolo, primul element ar fi dispărut din afișare.

E aceeași familie ca toate 🔴-urile tale de pe cifre: **capătul**. Primul element nu are un vecin în stânga, deci trebuie tratat separat.

FIX: afișezi `v[0]` înainte de buclă, apoi bucla pornește de la `i = 1`.

B26 — 5.5: prima treime pornește de la 1

`sortare-interval.h:133`

CUM E ACUM:

```
sortareTreiParti(9 7 5 3 1 2)  ->  9 7 3 5 1 2     trebuia 7 9 3 5 1 2
```

DE CE: `sortareInterval(v, 1, n / 3 - 1)`. Vectorul începe de la poziția `0`, deci prima treime e `0 .. n/3 - 1`. Pe `n = 6`, intervalul tău e `1 .. 1`, adică un singur element, iar `9` și `7` nu se ating. A doua și a treia treime le-ai pus corect, `n/3 .. 2*(n/3) - 1` și `2*(n/3) .. n - 1`, deci formula o ai. A greșit doar primul capăt.

FIX: `sortareInterval(v, 0, n / 3 - 1)`.

B15 — `produsCifre` nu poate întoarce un număr de două cifre (restanță din runda 3)

`parcurgerea-vectoriilor.h:183`

CUM E ACUM: `produsCifre(234)` → `4`, trebuia `24`. `produsCifre(25)` → `0`, trebuia `10`.

DE CE: `suma = suma * n % 10;`. `*` și `%` au **aceeași prioritate** și se evaluează de la stânga la dreapta, deci compilatorul citește `(suma * n) % 10`. Rezultatul e restul unei împărțiri la 10, deci **mereu între 0 și 9**. Ai copiat linia din `sumaCifrelor` (`suma + n % 10`, care e corectă), dar acolo `%` are prioritate mai mare decât `+` și parantezele implicite cad unde trebuie. La `*` cad invers.

FIX: `produs = produs * (n % 10);` (și numele acumulatorului: `produs`, nu `suma`).

B16 — „prime între ele" nu înseamnă „amândouă prime" (restanță din runda 3)

`parcurgerea-vectoriilor.h:223`

CUM E ACUM: `if(isPrim(v[i]) == true && isPrim(i) == true)`.

DE CE: „`a` și `b` sunt prime între ele" e o relație **între două numere**: nu au niciun divizor comun în afară de 1, adică `cmmdc(a, b) == 1`. Nu cere ca vreunul să fie număr prim. `8` și `9` sunt prime între ele, deși niciunul nu e prim. `3` și `9` nu sunt prime între ele, deși `3` e prim.

FIX: `cmmdc(v[i], i) == 1`. `cmmdc` l-ai reparat chiar în runda asta și acum merge pe toate cazurile, inclusiv `0`.

🟡 Importante

M13 — Ai rescris algoritmi care există deja în `algoritmi.h`. `cautare_binara.h:9` `cautareBinara1` ≡ `cautareBinara` · `cautare_binara.h:108` `inserareInVector` ≡ `inserareElement` · `sortare-interval.h:40-77` `sortareInterval` și `sortareIntervalDescrescator`, copiate întregi. Fișa zicea: „nu ai voie să rescrii un algoritm care există deja". Copia din `sortare-interval.h` mai are o problemă: dacă incluzi vreodată și `algoritmi.h`, primești `redefinition of 'sortareInterval'`. Pune `#include "algoritmi.h"` sus și șterge copiile.

M14 — Funcțiile care răspund la o întrebare schimbă vectorul primit. `mediana` (`sortare_bule.h:9`), `kValoare` (`:31`), `primeleK` (`sortare_selectie.h:10`) sortează direct vectorul cu care sunt chemate. Rulat: după `mediana(7 2 9 4 5)`, vectorul e `2 4 5 7 9`. Același mecanism ca la B23. Aici cerința nu interzice explicit, dar cine cheamă `mediana` nu se așteaptă să-și găsească datele amestecate. Fă o copie, sortează copia.

M15 — Ai pus trasări pe hârtie în `algoritmi.h`. `algoritmi.h:44-49`, `:137`, `:234-242`. Trasările sunt foarte bune (vezi mai jos), dar `algoritmi.h` e material de referință (B17). Mută-le în headerul tău, deasupra funcției tale care folosește algoritmul. La fel și schimbarea `mij = (sup + inf) / 2` de la `:59`: e corectă pentru vectorii tăi, dar forma originală, `inf + (sup - inf) / 2`, există ca să nu depășească `int` pe vectori foarte mari.

M16 — Cinci fișiere goale comise. `interclasare.h`, `munte_vale.h`, `stergere.h` (0 octeți), `sortate_bule.h` (doar garda, cu greșeală de tipar în nume: **sortate**) și `fisa2.h`, deschis încă din runda 1. Un fișier intră în git când are ceva în el.

M1 (runda 1) — garda cu cratimă, a treia oară. `sortare-interval.h:1`: `SORTARE-INTERVAL_H_INCLUDED`. Fișa avea regula scrisă în caseta de sus, iar în celelalte 4 headere noi ai respectat-o (`MINIM_MAXIM_H_INCLUDED` ✅). Rămân deschise și `while-ex.h:1`, `parcurgerea-vectoriilor.h:1`.

M4 (runda 1) — `cmmmc` depășește `int`. `while-ex.h:345`. Acum reprodus: `cmmmc(50000, 70000)` → `-79496`, trebuia `350000`. `a * b` se calculează înainte de împărțire. Împarte întâi: `a / cmmdc(a, b) * b`.

M17 — `cifraMaxima(0)` → `-1`. `while-ex.h:431`. Repararea lui B2 a mutat cazul `0` în afara buclei. `0` are o cifră, pe `0`.

Restanțe neatinse din runda 1: B3 (două bucle infinite, `while-ex.h:126`, `:171`), B6 (`eliminareCifra` scoate toate aparițiile), M2 (`% p` în loc de `% 10`, `:154`, `:201`), M3 (`pow` fără `<cmath>`, `:479`, `:554`), M6 (definiții în `.h`). Sunt toate în funcții vechi din `while-ex.h`, pe care nu le mai chemi. Rezolvă-le când termini secțiunile 1–5.

🟢 Cleanups

C20 — Nume care spun altceva decât face funcția: `afisareDistinceDescrescatoare` afișează **crescător**; `celMaiMicNrCuSumaCifrelorGresit` e varianta **corectă**, iar cea fără „Gresit" e cea veche, greșită. Șterge-o pe cea veche și dă-i noii numele bun.

C21 — `sol1`, `sol2`, `sol31`, `sol34`: câte un vector `v` declarat și nefolosit, lângă `v2`, cel folosit. Patru dintre warning-urile de la F9.

C22 — 4.1 cu `k` mai mare decât `dim`: `primeleK(v, 5, 7)` citește dincolo de cele 5 valori. Ori limitezi `k` la `dim`, ori scrii în comentariul de deasupra că `k <= dim` e obligatoriu.

C23 — Mesaje de commit: `clasa`, `commit clasa`, `commit tema 15.10`. Scrie ce conține: „5.1–5.5 sortare pe interval; 2.4 neterminat".

C18 (runda 3) — `bin/`, `obj/`, `.layout`, `.depend` tot urmărite în git. Fiecare commit al tău cară ~1 MB de `.exe`.

Ce e bine, și merită spus pe nume

Ai reparat B2 în toate cele trei locuri. Am scris-o mai sus, dar o mai scriu o dată: e cel mai important lucru din runda asta.

Un header pe fiecare algoritm, exact cum cerea fișa. `minim_maxim.h`, `cautare_binara.h`, `sortare_bule.h`, `sortare_selectie.h`, `sortare-interval.h`, fiecare cu `#include "algoritmi.h"` sus și cu enunțul exercițiului în comentariu deasupra funcției. Când deschizi un fișier, știi ce e în el.

1.1–1.3 sunt pe o linie fiecare, și sunt corecte.

```
return v[pozMaxim(v, dim)] - v[pozMinim(v, dim)];
```

Fără nicio buclă, refolosești ce ai. Ăsta e scopul fișei.

`sortarePeInterval` interschimbă `a` și `b` dacă sunt invers. Nu ți-a cerut-o nimeni. Rulat cu `(4, 1)` dă același rezultat ca `(1, 4)`. E primul caz-limită pe care l-ai tratat **înainte** să ți-l arăt eu.

`cateInterschimbari` (3.5) e singurul loc în care ai rescris bulele, și e locul în care trebuia. `sortareCrescator` nu numără nimic, deci trebuia să-ți faci propria variantă cu un contor. Rulat: `3 1 2` → `2`, `1 2 3` → `0`.

Trasările pe hârtie, a treia rundă la rând. `sortare-interval.h:8-38`: cinci treceri complete ale metodei bulelor, cu `i`, `aux`, `v[i]` și `v[i + 1]` pe fiecare pas. Asta e cea mai bună unealtă de învățare pe care o ai. Data viitoare, trasează și funcția **ta**, pe exemplul din fișă, nu doar algoritmul din `algoritmi.h`. Dacă trasai `sortareIntervalDescrescator` pe `9 7 5 3 1 2`, vedeai pe hârtie, la primul pas, cum dispare `3`.

`cautarePrimaPoz` e o căutare binară corectă, scrisă de la zero. Cu `<=`, cu `>=`, cu `poz = dim` ca valoare pentru „nu există". Rulat: corectă pe toate capetele.

Before / After

B17 + B18 — interschimbarea

Acum, `sortare-interval.h`:

```
void sortareIntervalDescrescator(int v[], int start, int finish)
{
    int aux;
    ...
            if (v[i] < v[i + 1])
            {
                v[i + 1] = aux;
                int aux = v[i];
                v[i] = v[i + 1];
                sortat = false;
            }
```

Cum ar trebui:

```
void sortareIntervalDescrescator(int v[], int start, int finish)
{
    ...
            if (v[i] < v[i + 1])
            {
                int aux = v[i];
                v[i] = v[i + 1];
                v[i + 1] = aux;
                sortat = false;
            }
```

Și mai bine: șterge funcția din `sortare-interval.h` de tot și pune `#include "algoritmi.h"` sus (M13). Acolo e deja scrisă corect, după ce pui linia înapoi.

B19 — `return` pe toate drumurile

Acum:

```
int pozitiaDeInserat(int v[], int dim, int key)
{
    for(int i = 0; i < dim; i++)
    {
        if(v[i] > key)
        {
            return i;
        }
    }
}
```

Cum ar trebui:

```
int pozitiaDeInserat(int v[], int dim, int key)
{
    for(int i = 0; i < dim; i++)
    {
        if(v[i] > key)
        {
            return i;
        }
    }
    return dim;
}
```

B20 — refolosește funcția corectă

Acum:

```
int ceaMaiMicaValoare(int v[],int dim, int key)
{
    int maxim= dim - 1;
    int minim = 0;
    int poz = dim;
    while(minim < maxim)
    {
        int mij = (minim + maxim) / 2;
        if(v[mij] > key)
        ...
```

Cum ar trebui:

```
int ceaMaiMicaValoare(int v[], int dim, int key)
{
    int poz = cautarePrimaPoz(v, dim, key);
    if(poz < dim)
    {
        return v[poz];
    }
    return -1;
}
```

B21 — întâi ordonezi

Acum:

```
if(cautareBinara(v, dim, x) != -1)
```

Cum ar trebui:

```
sortareCrescator(v, dim);
if(cautareBinara(v, dim, x) != -1)
```

B23 — fără sortare

Acum:

```
sortareCrescator(v, dim);
for(int i = 0; i < dim; i++)
{
    if(v[i] != aux[i])
```

Cum ar trebui:

```
for(int i = 0; i < dim - 1; i++)
{
    if(v[i] > v[i + 1])
    {
        return false;
    }
}
return true;
```

B24 — muți numerele, compari cifrele

Acum:

```
aux[i] = v[i] % 10;
...
sortareSelectie(aux, dim);
```

Cum ar trebui, în bucla de selecție scrisă de tine:

```
if(v[j] % 10 < v[pozMin] % 10)
{
    pozMin = j;
}
```

Q&A — răspunde în scris, înainte de runda 5

1. În `algoritmi.h`, compilatorul ți-a spus `undeclared identifier 'aux'`. În `sortare-interval.h`, ai adăugat `int aux;` și eroarea a dispărut. Explică de ce programul a mers **mai rău** după ce eroarea a dispărut: de unde vin numerele `16777216` din rezultat?

2. Pe `30 12 45 7 19`, căutarea binară îl găsește pe `30`, dar nu pe `19`. Trasează pe hârtie căutarea lui `19`: ce valoare e la mijloc, ce jumătate aruncă, și de ce n-are voie s-o arunce.

3. `pozMaxim(v, dim)` și `v[pozMaxim(v, dim)]`. Pe `3 3 8 1`, ce dă fiecare? Și de ce `aDouaValoare(3 3 8 1)` a dat totuși răspunsul corect?

Runda 3 — 2026-10-01 — sha ee7010e (condensată, nelivrată la timp)

Generată pe 2026-10-01, n-a ajuns la tine. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B15	`parcurgerea:183`	`produsCifre`: `suma * n % 10` → mereu o cifră	🔴 NEATINS (vezi runda 4)
B16	`parcurgerea:223`	„prime între ele” verificat cu `isPrim`	🔴 NEATINS (vezi runda 4)
B9	`parcurgerea:55–57`	`isPrim(i)` în loc de `isPrim(v[i])`	✅ ÎNCHIS

#	Loc	Constatare	Stare pe `6c66dca`
M10	`parcurgerea:109`	`afisareRasturnat` testează doar capetele	✅ ai ales palindromul: `punctul_c` cu `ePalindrom`
M11	`parcurgerea:156`	`sol1d` nu poate afișa ramura adevărată	🟡 NEATINS
M12	`parcurgerea:20`, `:253`	Cod comentat care nu compilează	🟡 `punctul_c` rescris ✅; `sol1` și `punctul_g` tot comentate
M7	—	`isPrim` ≡ `estePrim`, `CifraMinima` ≡ `cifraMinima`	🟡 NEATINS
C14	`parcurgerea:107`, `:207`	Variabile nefolosite	🟢 NEATINS
C15	`parcurgerea:168`	`?:` cu `<<`	🟢 NEATINS
C16	`parcurgerea:142`	`divizoriProprii` întoarce un număr	🟢 NEATINS
C17	—	`soll2`	✅ redenumit `sol2`
C18	`bin/`, `obj/`	Binare în git	🟢 NEATINS

Runda 2 — 2026-10-01 — sha fdda59a (condensată)

Textul integral e în PDF-ul rundei 2 (commit `b3b1bf6`).

#	Constatare	Stare pe `6c66dca`
B8	Nu compila, 7 erori	✅ ÎNCHIS (runda 3)
B9	`contorPrimeVector` număra indicii	✅ ÎNCHIS
B10	Indexare de la 1 vs de la 0	✅ ÎNCHIS
B11	Palindromul testa doar capetele	✅ ÎNCHIS
B12	`punctul_g` — buclă infinită	✅ ÎNCHIS
B13	`if` în afara buclei	✅ ÎNCHIS
B14	Funcție `int` fără `return`	✅ `afisareaElemVect` e `void` · 🔴 tiparul revine în runda 4 (B19)
M7	Funcții rescrise, existente deja	🟡 NEATINS · și în runda 4 (M13)
M8	`int c[10]` cu indexare de la 1	✅ codul a fost rescris
M9	`punctul_g` folosește `nr` nesetat	🟡 cod comentat
C7	`CIfraMinima`	✅ `CifraMinima`
C8	`//todo:` învechit	✅
C9	`dim` declarat, literal pasat	🟢 NEATINS (`parcurgerea:69`)
C10, C11	`main.cpp` vechi	✅ `main.cpp` rescris

#	Constatare	Stare pe `6c66dca`
C12	Binare în git	🟢 NEATINS
C13	Mesaje de commit	🟢 NEATINS

Runda 1 — 2026-09-24 — sha 2734bb4 (condensată)

Livrată pe 2026-10-01, odată cu runda 2. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B1	`main.cpp:1`	`while-ex.h` nu ajungea la compilator	✅ compilat separat de mine: 0 erori
B2	`while-ex.h:72`, `:429`, `:352`	Cifra scoasă înainte de `while`	✅ ÎNCHIS în toate cele 3 locuri
B3	`while-ex.h:126`, `:171`	Două bucle cu `n` nemodificat → nu se termină	🔴 NEATINS
B4	`while-ex.h:124`, `:169`	Funcții `int` fără `return`	🔴 2 rămase · vezi B19
B5	`while-ex.h:303`	`cmmdc(0, 0)` → împărțire la zero	✅ ÎNCHIS
B6	`while-ex.h:452`	`eliminareCifra` scoate toate aparițiile	🔴 NEATINS
B7	`while-ex.h:370`	`celMaiMicNrCuSumaCifrelor(20)` → 99	✅ ÎNCHIS (funcția nouă)
M1	gărzile `.h:1`	Cratimă în numele macro-ului	🔴 NEATINS + `sortare-interval.h`

#	Loc	Constatare	Stare pe `6c66dca`
M2	`while-ex.h:154`, `:201`	`% p` în loc de `% 10`	🔴 NEATINS
M3	`while-ex.h:479`, `:554`	`pow` fără `<cmath>`	🔴 NEATINS
M4	`while-ex.h:345`	`a * b` → depășire	🔴 reprodus: `cmmmc(50000,70000)` → `-79496`
M5	`fisa2.h`	Fișier gol	🔴 NEATINS · vezi M16
M6	toate `.h`	Definiții în `.h`	🟡 NEATINS
C1–C3	—	`dim`/literal, `int n` nefolosit, `isZero`	🟢 NEATINSE
C4	`bin/`, `obj/`	Binare în git	🟢 NEATINS
C5	`while-ex.h:8`, `main.cpp:3`	`//todo:` învechite	🟢 `main.cpp` ✅, `while-ex.h:8` rămas
C6	—	Mesaje de commit	🟢 NEATINS